C LAB PROGRAMS

Compiled & Tested — Online C Compiler Style

Compiler: GCC (GNU Compiler Collection), C11

The code/output panels below are formatted to resemble an online C compiler interface. They are presentation-style panels, not literal screenshots of a third-party website.

Question 1: First Palindromic String

Question: Refer to the corresponding problem statement in the submitted lab sheet.

Online C Compiler GCC • C11

Q1.c

```c
#include <stdio.h>
#include <string.h>

int isPalindrome(char s[]) {
    int i = 0, j = strlen(s) - 1;
    while (i < j)
        if (s[i++] != s[j--]) return 0;
    return 1;
}

int main() {
    char words[][30] = {"abc", "car", "ada", "racecar", "cool"};
    int n = 5;

    for (int i = 0; i < n; i++) {
        if (isPalindrome(words[i])) {
            printf("First palindromic string: %s\n", words[i]);
            return 0;
```

Output / Terminal

```
$ gcc Q1.c -o Q1 && ./Q1

First palindromic string: ada
```

Compilation successful • Program exited with code 0

Compiled output: First palindromic string: ada

Question 2: Count Common Elements in Two Arrays

Question: Refer to the corresponding problem statement in the submitted lab sheet.

Online C Compiler GCC • C11

Q2.c

```c
#include <stdio.h>

int exists(int a[], int n, int x) {
    for (int i = 0; i < n; i++)
        if (a[i] == x) return 1;
    return 0;
}

int main() {
    int nums1[] = {2, 3, 2};
    int nums2[] = {1, 2};
    int n = 3, m = 2, answer1 = 0, answer2 = 0;

    for (int i = 0; i < n; i++)
        if (exists(nums2, m, nums1[i])) answer1++;

    for (int i = 0; i < m; i++)
        if (exists(nums1, n, nums2[i])) answer2++;
```

Output / Terminal

```
$ gcc Q2.c -o Q2 && ./Q2

answer1 = 2
answer2 = 1

Compilation successful • Program exited with code 0
```

Compiled output: answer1 = 2
answer2 = 1

Question 3: Sum of Squares of Distinct Counts

Question: Refer to the corresponding problem statement in the submitted lab sheet.

Online C Compiler GCC • C11

Q3.c

```c
#include <stdio.h>

int main() {
    int nums[] = {1, 2, 1};
    int n = 3;
    long long sum = 0;

    for (int i = 0; i < n; i++) {
        int distinct[100], d = 0;
        for (int j = i; j < n; j++) {
            int found = 0;
            for (int k = 0; k < d; k++)
                if (distinct[k] == nums[j]) found = 1;
            if (!found) distinct[d++] = nums[j];
            sum += (long long)d * d;
        }
    }

```

Output / Terminal

```
$ gcc Q3.c -o Q3 && ./Q3

Sum of squares of distinct counts = 15
```

Compilation successful • Program exited with code 0

Compiled output: Sum of squares of distinct counts = 15

Question 4: Equal Values with Product Divisible by k

Question: Refer to the corresponding problem statement in the submitted lab sheet.

Online C Compiler GCC • C11

Q4.c

```c
#include <stdio.h>

int main() {
    int nums[] = {3, 1, 2, 2, 2, 1, 3};
    int n = 7, k = 2, count = 0;

    for (int i = 0; i < n; i++)
        for (int j = i + 1; j < n; j++)
            if (nums[i] == nums[j] && (i * j) % k == 0)
                count++;

    printf("Number of valid pairs = %d\n", count);
    return 0;
}
```

Output / Terminal

```
$ gcc Q4.c -o Q4 && ./Q4

Number of valid pairs = 4

Compilation successful • Program exited with code 0
```

Compiled output: Number of valid pairs = 4

Question 5: Find Maximum with Least Time Complexity

Question: Refer to the corresponding problem statement in the submitted lab sheet.

Online C Compiler GCC • C11

Q5.c

```c
#include <stdio.h>

int main() {
    int a[] = {1, 2, 3, 4, 5};
    int n = 5;
    int max = a[0];

    for (int i = 1; i < n; i++)
        if (a[i] > max) max = a[i];

    printf("Maximum element = %d\n", max);
    return 0;
}
```

Output / Terminal

```
$ gcc Q5.c -o Q5 && ./Q5

Maximum element = 5

Compilation successful • Program exited with code 0
```

Compiled output: Maximum element = 5

Question 6: Bubble Sort

Question: Refer to the corresponding problem statement in the submitted lab sheet.

Online C Compiler GCC • C11

Q6.c

```c
#include <stdio.h>

int main() {
    int a[] = {5, 1, 4, 2, 8};
    int n = 5;

    for (int i = 0; i < n - 1; i++) {
        int swapped = 0;
        for (int j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swapped = 1;
            }
        }
        if (!swapped) break;
    }

    printf("Sorted array: ");
```

Output / Terminal

```
$ gcc Q6.c -o Q6 && ./Q6

Sorted array: 1 2 4 5 8
Time Complexity: O(n^2)



Compilation successful • Program exited with code 0
```

Compiled output: Sorted array: 1 2 4 5 8
Time Complexity: O(n^2)

Question 7: Binary Search

Question: Refer to the corresponding problem statement in the submitted lab sheet.

Online C Compiler GCC • C11

Q7.c

```c
#include <stdio.h>

int main() {
    int arr[] = {-9, 3, 4, 6, 8, 9, 10, 30};
    int n = 8, key = 10;
    int low = 0, high = n - 1, found = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == key) {
            found = mid;
            break;
        } else if (arr[mid] < key)
            low = mid + 1;
        else
            high = mid - 1;
    }

```

Output / Terminal

```
$ gcc Q7.c -o Q7 && ./Q7

Element 10 is found at position 7
Time Complexity: O(log n)
```

Compilation successful • Program exited with code 0

Compiled output: Element 10 is found at position 7
Time Complexity: O(log n)

Note: Binary search requires a sorted array. After sorting the given values, 10 is at 1-based position 7.

Question 8: Merge Sort in O(n log n)

Question: Refer to the corresponding problem statement in the submitted lab sheet.

Online C Compiler GCC • C11

Q8.c

```c
#include <stdio.h>

void merge(int a[], int l, int m, int r) {
    int i = l, j = m + 1, k = 0, temp[100];
    while (i <= m && j <= r)
        temp[k++] = (a[i] < a[j]) ? a[i++] : a[j++];
    while (i <= m) temp[k++] = a[i++];
    while (j <= r) temp[k++] = a[j++];
    for (i = l, j = 0; i <= r; i++, j++) a[i] = temp[j];
}

void mergeSort(int a[], int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}
```

Output / Terminal

```
$ gcc Q8.c -o Q8 && ./Q8

Sorted array: 1 2 3 5 8
Time Complexity: O(n log n)



Compilation successful • Program exited with code 0
```

Compiled output: Sorted array: 1 2 3 5 8
Time Complexity: O(n log n)